

CSE 211: Data Structures

2005/2006-I

☺

MIDTERM

(Open Book Exam) 90 min.

1	
2	
3	
4	

28.11.2005

Name :
Student id :
Section :

1. Answer the following questions:

a) [15] What is the execution time of the piece of code on the leftside, in terms of n? Justify your answer.

```
long n; // input size
...
k=n;
while (k>=1) {
    i=1;
    while (i<=k) {
        ...
        i*=3;
    }
    k/=2;
}
```

Hint: $\lg(x/y) = \lg(x) - \lg(y)$

ANSWER:
Assuming $n=2^m$
First loop will iterate m times, at each time inner loop will iterate depending on k values

m	k	number of iterations of the inner loop
1	n	$\log_3(n) \sim \lg(n)$
2	n/2	$\log_3(n/2) \sim \lg(n/2)$
3	n/4	$\log_3(n/4) \sim \lg(n/4)$
…	…	…
m	1	$\log_3(n/2^m) \sim \lg(n/2^m)$

As a total:

$$\sum_{i=0}^{m} \lg(n/2^i),$$

$$\sum_{i=0}^{m} \lg(n/2^i) = \sum_{i=0}^{m} (\lg(n) - \lg(2^i))$$

$$= \lg(n)\sum_{i=0}^{m} 1 - \sum_{i=0}^{m} i$$

$$= \lg(n)m - m(m-1)/2, m = \lg(n)$$

$$= \lg(n)\lg(n) - \lg(n)(\lg(n)-1)/2$$

$$= O(\lg^2(n))$$

b) [10] Write the postfix form of ((x+y)*2+(324–d))*E :
x y + 2 * 324 d - + E *

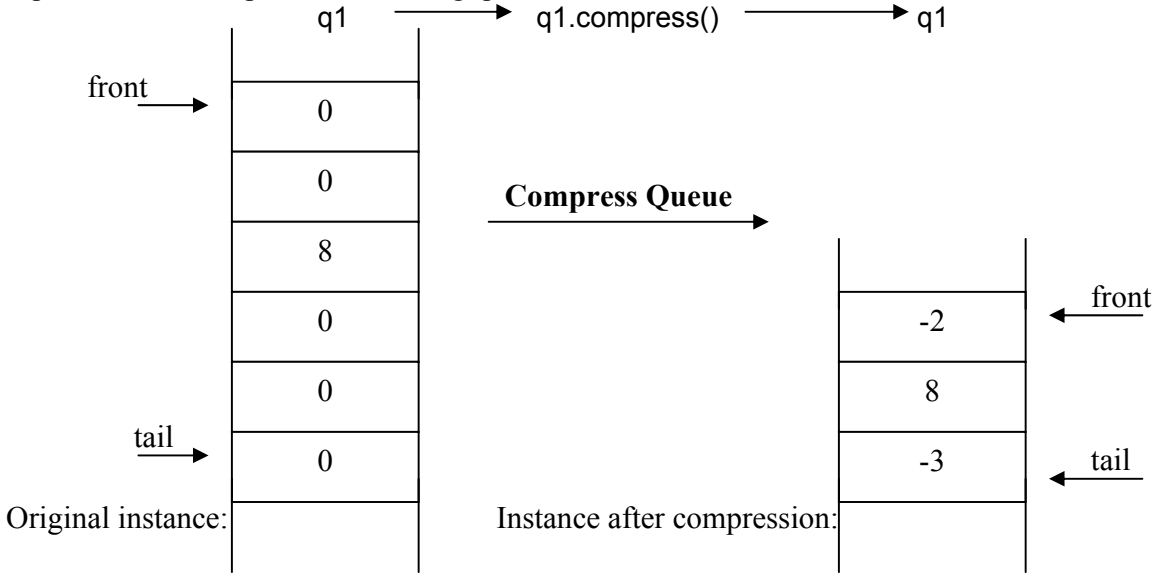
2. [30] Consider a C++ library that supports Queues as a data type. That is, one can declare

Queue <int>q1;

and the following operations are defined:

```
template <class myType>
class Queue {
  …
  public:
    // inserts an object (of myType) in the rear of a Queue
    void enqueue( myType obj );
    // looks at the object in front of this queue without
    // removing it.
    myType peek();
    // removes the object in front of this queue
    // and returns the object
    myType dequeue();
    // returns true if the queue is empty, 0 otherwise
    bool isEmpty();
}
```

You have discovered that in your program all of the queues contain non-negative integers and additionally more than %65 of the items in the queues are zeros. So you come up with the idea for compressing these queues, taking up less space: each contagious sequence of queue items having 0 is replaced by a negative number, whose absolute value is the total number of zeros in that sequence. For example, the following queue is:



Write **void compress**() as an additional member function for the Queue class defined above to self compress an original queue as explained above. Your solution should run in linear time on the length of the resulting queue and should be based on available public methods in the Queue class only. Consider all speacial cases, e.g., empty queue, queue full of zeros only,etc... *Hint*: You can use as many Queue objects as needed, as a temporary storage.

```
// "this->" can be omitted
void compress(){
     Queue <int> tmp;
     int val, zeroCounter=0;

     while(!this->isEmpty()){
          val = this->dequeue();

          while(val && ! this->isEmpty()){ // val!=0
               tmp.enqueue(val);
               val = this->dequeue();
          }
          while(!val && ! this->isEmpty()){ // val==0
               zeroCounter--;
               val = this->dequeue();
          }
          if (zeroCounter){
               tmp.enqueue(zeroCounter);
               zeroCounter=0;
          }
     }

     while(!tmp.isEmpty()){
          val = tmp.dequeue();
          this->enqueue(val);
     }

}
```

3. [20] Consider the following matrix *m*

column index values (j) ↓

row index values (i) →

i \ j	1	2	3	4	5
1	3	5	7	9	11
2	5	8	11	14	17
3	7	11	15	19	23
4	9	14	19	24	29
5	11	17	23	29	35

Assume that in an application, you need *m*(i,j) values. How can you represent *m* most efficiently (i.e., allocating the minimum amount of memory space with no dramatical changes in access time) in your code? Show all your work.

ANSWER:
No memory is required, m(i,j) can be implemented as a function, returning the related value:
m(i,j) = (i+1)*j+i
OR
m(i,j) = (j+1)*i+j

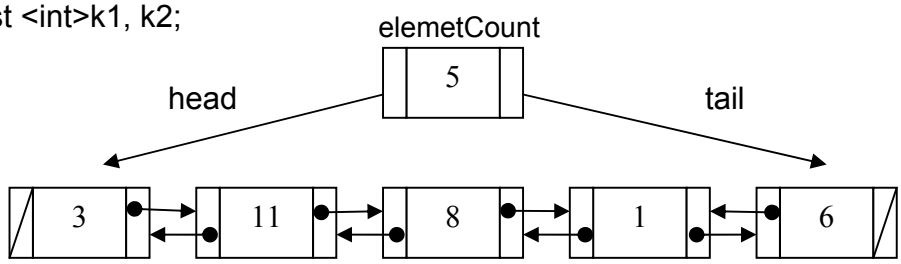
[25] Classes node and linkedlist is defined as follows:

```
template<class T> class node
{
 public:
    node<T> *next;
    node<T> *prev;
    T data;
};
```

```
template<class T> class linkedlist
{
 public:
  bool insertHead(const T &x);
  bool deleteHead(T &x);
  bool insertTail(const T &x);
  bool deleteTail(T &x);
  void clear();
  bool isEmpty() const;
  int getElementCount() const;
private:
  node<T> *head;
  node<T> *tail;
  int elementCount;
}
```

Assume that in an application, you are holding a linked list of UNIQUE items. You need a method to locate a given key value and split the linked list into two linked lists, destroying the original. First part should be kept in the self linked list, the second part should be returned in a new linkedlist object, including the seperator key value.
For example, assume a linked list holding unique integer values, denoted as k1:
linkedlist <int>k1, k2;

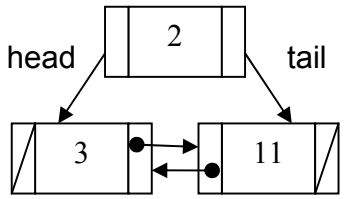
k1:



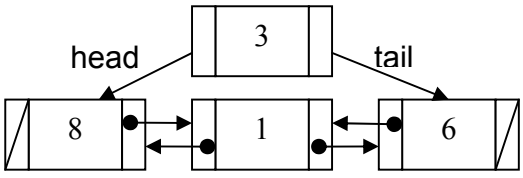
k2: empty linked list

a call to **k1.split(8, k2)** returns true and updates k1 and k2 as follows

k1:



k2:



Write **bool split(T key, linkedlist &k2)** as an additional member function for the linkedlist class as described above. Method should return true if key is found, false otherwise.

```
// "this->" can be omitted
bool split(T key, linkedlist &k2)
{
     node *curr = this->head;
     bool found=false;
     int i, cnt=0;
     T tmp;

     k2.clear(); // just in case k2 is not empty

     // search the key
     while(curr && !found){
          if (curr->data==key)
               found = true;
          else{
               curr = curr->next;
               cnt++;
          }
     }

     // if the key is found, perform the split operation
     if (found){
          // location for splitting from the tail
          cnt = this->elementCount - cnt;

          for (i=0;i<cnt;i++){
               this->deleteTail(tmp);
               k2.insertHead(tmp);
          }
     }
     return found;
}
```